command and query separation (CQS) principle. This states that each method should be either a command that performs an action or a query that returns data to the caller, but not both. In other words, asking a question should not change the state of things. More formally, methods should return a value only if they are referentially transparent and therefore have no side effects. It has been argued that while CQRS is a useful principle, it is not sufficient when dealing with data structures such as stacks or queues because when reading from a stack, the element must be removed, causing side effects.

Even if CQS is not always applicable, it is a powerful idea to which CQRS adds even more. The basic idea is that these commands and queries should be executed not on one but on two different objects—one for the write side and one for the read side. For this, you need different models that exist in different places. For example, the optimization for writing can be a ledger or table normalized to the NF3 degree, and the optimization for reading can be a table normalized to the NF1 degree.

The CQRS abstraction plays well with event sourcing. When you combine the two, the models are no longer stored in two places; only the events are persisted. But you still have two places: one to send queries and another to send commands. For example, with Akka Persistence, there is an entity that you send commands to that keeps its state in memory, and the entity produces events that are persisted in the journal to which you send the queries.

B.2.3 Business abstractions

DDD’s abstractions give you clarity about how to define the boundaries of the data as it relates to the business. Perhaps the most